

Why This Course Requires an AI Coding Assistant

This course requires every student to use an AI Coding Assistant throughout the semester. This is not a convenience — it is a deliberate professional choice.

Nuclear cybersecurity practitioners at national laboratories, utilities, and regulatory agencies are already using AI-assisted tools to draft threat models, analyze regulatory documents, triage incidents, and produce security assessments at greater depth than was previously possible. A graduate course that ignores this reality would be preparing you for a profession that no longer exists.

More importantly: the capabilities covered in this document — thinking modes, function calling, structured output, multimodal analysis, and MCP tool integration — are not features. **They are the skills this course builds.** Each workshop is designed around one or more of these capabilities applied to a real nuclear security problem.

Choosing Your Tool

Selecting your AI Coding Assistant is the first real decision of this course — and it is itself an assignment. You will evaluate and justify your choice based on the capability dimensions below. You may use one tool for the entire semester or combine tools by task; either approach is valid if you can defend it.

Evaluate any candidate tool against these capability areas:

- **Thinking & Reasoning Depth** — Does it support configurable reasoning modes (deep vs. fast)? Can it reason between tool calls? Does it preserve reasoning across turns?
- **Context Length** — How much can it hold in a single session? Loading full regulatory documents requires significant context capacity.
- **Function Calling & Tool Use** — Can it invoke external tools, APIs, or scripts during a session? This is essential for live data workflows.
- **Structured Output** — Can it reliably return JSON or other defined formats? Required for compliance matrices and threat models.
- **Vision & Multimodal Input** — Can it analyze images, diagrams, and PDFs? Critical for architecture review and document analysis.
- **MCP / Tool Integration** — Does it support Model Context Protocol servers for web search, document reading, and live data access?
- **Provenance & Trust** — Who built it, where is data processed, and is it appropriate for use with sensitive security content?

The last criterion is non-negotiable in nuclear security contexts. Only tools developed and hosted by US-based providers with clear data handling policies are appropriate for this course.

Regardless of tool, you are responsible for the accuracy of everything you submit. Verify all regulatory citations, technical claims, and system-specific assertions against primary sources.

Core Capabilities — The Skills You Are Building

The following capabilities are not optional features to explore. They are the technical foundation of AI-assisted security work. You will use each of them in the course workshops.

1. Thinking Modes & Deep Reasoning

Modern AI coding assistants support configurable reasoning depth. Understanding when and how to engage deep thinking is itself a professional skill.

What it is: Deep Thinking enables Chain-of-Thought reasoning — the model works through a problem systematically before producing an answer. This significantly improves accuracy on complex, multi-step analysis tasks.

Modes to look for in your chosen tool:

- **enabled** — Model auto-decides when to think deeply (default)
- **disabled** — Direct answers, lower latency, for simple queries
- **Interleaved Thinking** — Reasoning occurs between tool calls, allowing the model to interpret each result

before deciding the next step

- **Preserved Thinking** — Retains reasoning from prior turns; default on Coding Plan endpoints; critical for multi-session workflows
- **reasoning_effort** — Scales from **none** to **max**; use **max** for threat modeling, regulatory analysis, and complex incident scenarios

Nuclear security application: Enable deep thinking (**max effort**) for threat modeling, attack chain analysis, and regulatory gap assessment. Disable for simple lookups, format conversions, or quick summaries. Misjudging this wastes tokens and time — or produces shallow analysis on complex problems.

Critical implementation note: When using preserved or interleaved thinking, always return the complete, unmodified **reasoning_content** back to the model. Truncating it degrades performance and breaks cache efficiency.

2. Function Calling & Tool Use

Function calling allows the AI to interact with external systems — databases, APIs, file systems, and security tools — during a session. This transforms the model from a text generator into an active analysis agent.

What it is: You define callable functions (tools) with names, descriptions, and parameter schemas. The model decides when to invoke them, passes structured arguments, receives results, and reasons over the output before continuing.

Security-relevant applications:

- Query a CVE database during a vulnerability analysis session
- Execute a network scan script and have the model interpret results
- Pull NRC regulatory text from an API and map it to a control architecture
- Retrieve ICS/SCADA log data and run anomaly detection routines
- Chain multiple operations: search → analyze → draft recommendation

Best practices: Single responsibility per function. Validate all inputs. Log all calls. Implement permission controls. Never pass unvalidated user input directly to a function that executes code or queries a database.

Nuclear security application: In the Monitoring & Anomaly Detection workshop, you will build a function-calling workflow that queries a simulated ICS data stream, flags anomalies, and drafts an incident notification. This is the same pattern used in real SOC environments.

3. Structured Output

Structured output forces the model to return data in a defined format — typically JSON — rather than free-form prose. This is essential when your output needs to be machine-readable, consistently formatted, or fed into another system.

What it is: Set **response_format**: `{"type": "json_object"}` and define your expected schema in the system prompt. The model guarantees output conformance, enabling reliable programmatic processing.

Security-relevant applications:

- Threat models as structured JSON (actor, vector, likelihood, impact, mitigation)
- Compliance matrices with pass/fail fields per regulatory requirement
- Incident timelines with typed event fields
- Vulnerability assessments with CVSS-style scoring fields
- Configuration audit results with remediation status flags

Best practices: Start with simple schemas. Name fields explicitly. Add descriptions for each field in the system prompt. Implement validation against the schema after receiving output — JSON mode reduces but does not eliminate malformed responses in complex scenarios.

Nuclear security application: Your compliance matrix workshop (NRC 10 CFR 73.54) will require structured output. A JSON-formatted compliance matrix can be imported directly into a spreadsheet or ticketing system — exactly how this work is done at a licensed facility.

4. Vision & Multimodal Analysis

Vision Language Models (VLMs) accept images, diagrams, documents, and in some cases audio and video as inputs alongside text. This capability is particularly powerful for security work where evidence is rarely just text.

Modalities to look for in your chosen tool:

- **Images** — Architecture diagrams, network maps, screenshots, scanned documents
- **Files / PDFs** — Regulatory documents, configuration exports, inspection reports
- **Audio** — Transcription of interviews or practitioner briefings
- **OCR** — Structured text extraction from scanned documents

Security-relevant applications:

- Upload a network architecture diagram and ask the model to identify attack surfaces and missing segmentation controls
- Feed a scanned ICS schematic and map it to the Purdue Model zones
- Analyze a screenshot of anomalous SCADA readings against baseline
- Extract structured data from a scanned NRC inspection report

Nuclear security application: In the Physical Protection workshop, you will upload a facility diagram and use a VLM to identify cyber-physical intersection points — where a digital compromise could affect a physical safety boundary.

5. Model Context Protocol (MCP) Integration

MCPs extend the model's reach beyond its training data by connecting it to live external tools and data sources during a session.

Common MCP servers to look for:

- **Web Search** — Real-time queries for current threat intelligence, recent CVEs, and regulatory updates
- **Web Reader** — Full-page content extraction from URLs; feed the model a live NRC advisory or CISA alert directly
- **Vision** — On-demand image analysis within a session
- **Document Reader** — Structured extraction from PDFs and reports

Security-relevant applications:

- Pull the latest CISA ICS advisory and have the model assess applicability to a specific nuclear plant architecture
- Search for recent threat actor TTPs and incorporate them into a live threat model
- Read an NRC regulatory document and extract all applicable control requirements automatically

Nuclear security application: The Insider Threat workshop will use Web Search + Web Reader MCPs to pull current case studies and have the model synthesize a threat profile grounded in recent real-world incidents — not just training data.

Context Length & Session Management

Leading AI coding assistants support up to **1 million tokens of context**. This is not a trivial capability — it means you can load entire regulatory frameworks, multi-document collections, or long session histories into a single working context. Used well, it eliminates the need to re-explain your problem across sessions.

Practical guidance:

- Load full regulatory documents at session start rather than pasting excerpts
- Keep your session log as a running context — preserved thinking benefits from continuity across a multi-step analysis
- Be aware that very large contexts increase latency and token cost; use context caching where available
- Maximum output tokens vary by model tier — plan your prompts accordingly for long deliverables like compliance matrices or threat models

Academic Integrity in an AI-Assisted Course

Using an AI Coding Assistant is **required**, not prohibited. However, the following remain your sole responsibility:

- The accuracy of all regulatory citations and technical claims
- The analysis, judgment, and conclusions in your work products
- Understanding your submission well enough to discuss and defend it

Session logs are submitted with every assignment. They make your reasoning process visible. If you cannot explain or defend your submitted work in a follow-up discussion, it will not be counted.

Building Your Portfolio on GitHub

Every workshop produces a real security artifact. Over the semester you will build: threat models, access control policies, protocol analyses, compliance matrices, monitoring plans, an incident response package, and a capstone project.

Repository structure:

- `/workshops` — one folder per module; artifact + session log
- `/project` — semester project materials and interim drafts
- `/written` — ethics paper and final written analysis
- `README.md` — professional summary of the portfolio

A well-maintained repository is something you can link from a resume or share directly with a hiring manager. The combination of security artifacts and AI-assisted workflows is a differentiator that very few candidates currently have.

CS 581 — Nuclear Cybersecurity — Christopher Spirito — cspirito@boisestate.edu — Fall 2026